Эмулятор командной оболочки

Практическая работа №1 по конфигурационному управлению. Группа ИКБО-62-25, вариант 7. Реализация на Python с GUI на Tkinter.

Репозиторий: https://github.com/Gudrikh/shell-emulator

PDF-версия этого документа: README.pdf.

Требования и запуск

Нужен Python 3.10 или новее с Tkinter. Сторонние пакеты для запуска и unit-тестов не используются. Команды PowerShell выполняются из корня проекта, например `C:\shell-emulator`.

```
.\run.bat
```

По умолчанию загружается `data/vfs-stage4.csv`. В окне показаны доступные команды; заголовок содержит имя пользователя и имя компьютера.

Пример запуска с параметрами:

```
python -m src.main --vfs "data/vfs-stage4.csv" --prompt "DEMO> "
```

Запуск со стартовым скриптом:

```
.\scripts\test_stage4.bat
```

Отдельные ссылки на этапы

Каждая ссылка ведёт к состоянию проекта после соответствующего этапа. Ранее созданные коммиты сохраняют первоначальную реализацию. Доработки по замечаниям находятся в актуальной версии репозитория.

Этап	Содержание	Состояние проекта
1	REPL и GUI	Этап 1
2	Конфигурация	Этап 2
3	VFS из CSV	Этап 3
4	Основные команды	Этап 4
5	Изменение VFS	Этап 5

Этап 1. REPL

Реализованы окно, поле ввода, вывод команд и результатов. Заголовок формируется из реального имени пользователя и имени компьютера. Парсер раскрывает переменные окружения, например `$USERNAME` в Windows. Первоначальные `ls` и `cd` являлись заглушками; `exit` завершал работу. Неизвестные команды и незакрытые кавычки дают понятные ошибки.

Этап 2. Конфигурация

Параметры командной строки:

- `--vfs` - физический путь к CSV-файлу VFS;
- `--prompt` - пользовательское приглашение перед командой;
- `--script` - физический путь к стартовому скрипту.

Параметры печатаются в терминале при запуске. Стартовый скрипт выполняет команды последовательно и показывает ввод и вывод в окне. Пустые строки и комментарии с # пропускаются. Ошибка отдельной команды не мешает следующей; ошибка чтения скрипта выводится в окне.

Этап 3. VFS

CSV загружается в память. Поля идут строго в следующем порядке:

```
path,type,content_base64
```

- `path` - относительный путь с разделителем `/`;
- `type` - `dir` для папки или `file` для файла;
- `content_base64` - байты файла, закодированные Base64; для папок поле пустое.

Родительские папки нужно явно записать в CSV. Корень `/` существует неявно. Проверяются заголовки, число полей, пути, типы, Base64, дубликаты и существование родительских папок. Ошибка загрузки останавливает запуск до создания GUI.

Подготовлены минимальная VFS, несколько файлов, дерево с тремя уровнями, файлы для проверки команд и CSV с неверным форматом.

Этап 4. Основные команды

- `ls [-a] [-l] [-R] [путь]` - показывает файл или содержимое папки;
- `cd [путь]` - меняет текущую папку; без пути возвращает в корень VFS;
- `whoami` - выводит имя пользователя реальной ОС;
- `head файл` - выводит первые 10 строк файла в UTF-8;
- `head -n число файл` - выводит заданное неотрицательное число строк;
- `exit` - завершает программу; аргументы не допускаются.

Пути могут быть относительными или абсолютными внутри VFS. Поддерживаются `.` и `..`, выход выше корня запрещён.

Ключи `ls`:

- `-a` - включает скрытые имена, начинающиеся с точки;
- `-l` - выводит тип, размер содержимого в байтах и имя; размер содержимого папки равен нулю;
- `-R` - обходит вложенные каталоги и подписывает каждый раздел путём.

Ключи можно объединять: `ls -alR` равнозначно `ls -a -l -R`. Маркер `--` завершает разбор ключей и позволяет передать имя с дефисом. Поддерживается один путь на вызов. Неизвестные ключи вызывают ошибку. Подробный вывод ограничен метаданными CSV: права и время изменения в этой VFS не хранятся.

Примеры команд в окне эмулятора:

```
ls -a
ls -l docs
ls -R /
ls -alR docs
cd docs
head -n 2 sample.txt
cd
whoami
exit
```

Этап 5. Изменение VFS

- `mkdir путь` - создаёт одну папку; её родитель должен существовать;
- `mv источник назначение` - перемещает или переименовывает объект.

Если назначение - существующая папка, объект переносится в неё с прежним именем. Папка переносится вместе с потомками; текущий каталог обновляется, если он расположен внутри перемещённой папки.

Запрещены перезапись существующего объекта, перенос корня и перемещение папки внутрь себя. При ошибке дерево сохраняется. Все изменения остаются только в памяти. CSV на диске не перезаписывается; после нового запуска VFS снова загружается из исходного файла.

Проверки

Автоматические проверки:

```
python -m unittest discover -v
python scripts/check_quality.py
```

Тесты проверяют команды, ключи `ls`, ошибки CSV, перемещение дерева, обновление текущей папки и неизменность исходного CSV. Проверка качества ограничивает строки Python 80 символами, оценивает число ветвлений функций (не более 10) и запрещает числовые литералы в сравнениях исходного кода. Проверка рассчитана на `src/*.py`.

Сценарии демонстрации в GUI:

```
.\scripts\test_config.bat
.\scripts\test_config_error.bat
.\scripts\test_vfs_minimal.bat
.\scripts\test_vfs_many.bat
.\scripts\test_vfs_tree.bat
.\scripts\test_stage4.bat
.\scripts\test_stage5.bat
.\scripts\test_ls_options.bat
```

Сценарии этапов 4 и 5 и ключей `ls` заканчиваются ошибочной командой `exit now`, чтобы окно оставалось открытым для просмотра результатов. После проверки введите `exit` без аргументов в поле ввода.

Ошибка загрузки VFS:

```
python -m src.main --vfs "data/not-found.csv"
python -m src.main --vfs "data/vfs-invalid.csv"
```

Структура проекта

- `src/main.py` - GUI, параметры запуска, выполнение стартового скрипта;
- `src/shell.py` - разбор и выполнение команд, текущая папка VFS;
- `src/vfs.py` - загрузка CSV, проверки и хранение объектов в памяти;
- `data/` - примеры VFS;
- `scripts/` - стартовые сценарии, BAT-запуски, проверка качества;
- `tests/` - unit-тесты;
- `run.bat` - обычный запуск;
- `README.md` и `README.pdf` - инструкция и ссылки на этапы.